

五一数学建模竞赛

承 诺 书

我们仔细阅读了五一数学建模竞赛的竞赛规则。

我们完全明白，在竞赛开始后参赛队员不能以任何方式（包括电话、电子邮件、网上咨询等）与本队以外的任何人（包括指导教师）研究、讨论与赛题有关的问题。

我们知道，抄袭别人的成果是违反竞赛规则的，如果引用别人的成果或其它公开的资料（包括网上查到的资料），必须按照规定的参考文献的表述方式在正文引用处和参考文献中明确列出。

我们郑重承诺，严格遵守竞赛规则，以保证竞赛的公正、公平性。如有违反竞赛规则的行为，我们愿意承担由此引起的一切后果。

我们授权五一数学建模竞赛组委会，可将我们的论文以任何形式进行公开展示（包括进行网上公示，在书籍、期刊和其他媒体进行正式或非正式发表等）。

参赛题号（从 A/B/C 中选择一项填写）： B

参赛队号： T3837579826045

参赛组别（研究生、本科、专科、高中）： 本科

所属学校（学校全称）： 北京林业大学

参赛队员： 队员 1 姓名： 周济坤

队员 2 姓名： 董茹萍

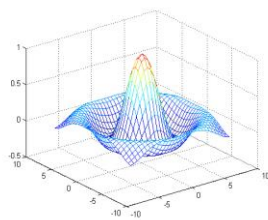
队员 3 姓名： 潘显雨

联系方式： Email: 1794680709@qq.com 联系电话： 18800154223

日期： 2024 年 5 月 4 日

（除本页外不允许出现学校及个人信息）

五一数学建模竞赛



题目：基于最短路径与线性规划的交通可达率优化方案

关键词：网络图模型 最短路径算法 线性规划模型 随机扰动

摘要：

在未来新城背景下，自动驾驶车辆能够自动遵循预设路线完成从指定起点到指定终点的出行。我们的任务是在给定的交通网络中，根据起点到终点的交通需求，完成各(起点，终点)对间的交通路径规划及相应的交通需求分配，使得当网络中某条或某几条路段出现突发状况时，网络中所有的交通需求的期望可达率最大。

最大化网络中所有交通需求的期望可达率，显然这是一个规划类问题。我们通过构建网络图模型，列写目标函数、约束条件、与决策变量，发现其线性可解。其中，通过最短路径算法，我们得到了目标函数的系数向量，通过附件数据，我们得到了等式约束与不等约束的系数矩阵，再加之非负约束，我们将所有参数带入 MATLAB 中的 `linprog` 函数中，成功求解一至三问。

在问题一中，我们给(1,9)规划路径 $1 \rightarrow 4 \rightarrow 7 \rightarrow 8 \rightarrow 9$ ，分配交通量180，给(3,7)规划路径 $3 \rightarrow 2 \rightarrow 1 \rightarrow 4 \rightarrow 7$ ，分配交通量140；在问题二中，我们给(27,6)规划路径 $27 \rightarrow 35 \rightarrow 28 \rightarrow 8 \rightarrow 9 \rightarrow 5 \rightarrow 6$ ，分配交通量121，给(19,25)规划路径 $19 \rightarrow 10 \rightarrow 41 \rightarrow 35 \rightarrow 27 \rightarrow 30 \rightarrow 25$ ，分配交通量115；在问题三中，我们给(32,39)规划路径 $32 \rightarrow 24 \rightarrow 7 \rightarrow 12 \rightarrow 11 \rightarrow 10 \rightarrow 16 \rightarrow 18 \rightarrow 2 \rightarrow 15 \rightarrow 13 \rightarrow 39$ ，分配交通量65，给(17,8)规划路径 $17 \rightarrow 14 \rightarrow 13 \rightarrow 15 \rightarrow 2 \rightarrow 18 \rightarrow 16 \rightarrow 10 \rightarrow 9 \rightarrow 8$ ，分配交通量63。其中，问题二相较于问题一扩大了网络的规模，强化了突发状况出现的条件；问题三相较于问题二给各路段增添了容量上限，需要将路段约束转化为了路径约束。

对于问题四，我们通过分析最短路径与最大化期望可达率的关系，筛选出 19 条最有价值路段，其中，有 4 条路段在多条路径规划中反复出现，我们将其视为交通“枢纽”，作为我们遍历集合中的固定项，再从剩余 15 条最有价值路径中随机抽取 2 条，作为我们遍历集合中的随机项，这样做既减少了运行开销，也保证了不会因减少遍历组合数而导致期望可达率降低。

最终我们得到了包括 $\{7 \rightarrow 26, 19 \rightarrow 11, 3 \rightarrow 6, 17 \rightarrow 13, 15 \rightarrow 3, 41 \rightarrow 9\}$ 在内的 5 组最优新修路段集，最高可达率达到了92.97%。

一、问题重述

在未来新城背景下，自动驾驶车辆能够自动遵循预设路线完成从指定起点到指定终点的出行。从指定起点到指定终点的交通量叫做交通需求，在多数情况下，我们需要在起点和终点间划分若干条路径，以实现交通需求的分配。当道路完全畅通时，起点到终点的交通可达率为百分之百，当某条或某几条路段出现突发状况时，交通可达率将因车辆无法顺利抵达终点而有所降低。

我们的任务是在给定的交通网络中，根据起点到终点的交通需求，完成各(起点，终点)对间的交通路径规划及相应的交通需求分配，使得当网络中某条或某几条路段出现突发状况时，网络中所有的交通需求的期望可达率最大。

二、问题分析

2.1 对问题一的分析

最大化网络中所有交通需求的期望可达率，显然这是一个规划类问题，求解规划类问题，首先应该明确目标函数、决策变量、与约束条件。附件 1 中给出了交通网络二指定的(起点，终点)对以及对应的交通需求量，我们的任务是完成各(起点，终点)对间的交通路径规划及相应的交通需求分配。对于交通路径规划，我们首先应该建立某种算法，以寻求起点到终点间合适的路径方案。我们注意到，题目中明确指出“优先选择距离短的路径”，于是在本文中我们将以“最短路径或较短路径”为目标，来规划各(起点，终点)对间的交通路径。而对于“如何在给定路径下，完成交通需求的分配”，我们发现题目恰好符合线性规划求解的条件，即目标函数与约束条件均为线性等式或不等式。我们选定分配到规划好的各条路径上的交通量作为线性规划求解的决策变量，根据交通网络二构建赋权有向图，将目标函数与约束条件输入 MATLAB 中的 linprog 函数，即可完成问题一的求解。

2.2 对问题二的分析

问题二相较于问题一有两点升级，一是扩大了网络的规模，二是强化了突发状况出现的条件。思路依然采用“以最短路径规划路径方案，以线性规划分配交通需求”，通过改变最短路径算法与线性规划的目标函数、决策变量、与约束条件，完成问题二的求解。

2.3 对问题三的分析

问题三相较于问题二有一点升级，即给各路段增添了容量上限，由于在我们的线性规划模型中，路径是决策变量，进一步将决策变量从路径细分为路段将大大增加模型复杂度与运行开销，于是我们先将路段约束转化为了路径约束，并将其加入到我们线性规划模型的约束条件中，完成了问题三的求解。

2.4 对问题四的分析

问题四要求在网络中增添 6 条新修路段，但由于这样的遍历组合过多，我们选择通过人工筛选的方式，先将“最有价值”的路段筛选出来，以此缩小我们的遍历集。然后每次从遍历集中抽取 6 条新修路段，组成新的网络，再由新的网络算出新的目标函数、决策变量、与约束条件，代入到线性规划模型中求解。最终根据可达率决出前五的路段，作为我们问题四的结果。

三、模型假设

3.1 题给假设

- 假设每个(起点，终点)对之间使用的路径数不超过 5，各路段长度均为单位

1, 优先选择距离短的路径。

- **假设**交通网络中所有车辆均为无人驾驶车辆, 并且所有车辆都服从系统预先规划的路径进行出行。

- **假设**交通流量值可以为任意的非负实数。

3.2 新增假设

- **假设**在大规模网络求解时, 以最短路径集或较短路径集替代起点到终点间所有可能的路径集。

四、符号说明

符号	含义
D	赋权有向图 $\Leftrightarrow$ 网络
v	顶点 $\Leftrightarrow$ 节点
a	弧 $\Leftrightarrow$ 路段
p	路径
W	邻接矩阵
(s, t)	(起点, 终点)对
$P_{(s,t)}$	从起点 s 到终点 t 的所有可能路径
$x_p^{(s,t)}$	(s, t) 中分配到路径 p 上的交通量
$Total_{(s,t)}$	(s, t) 的总交通量
n	突发状况的可能数
y_p^i	关于 p 是否出现突发状况的判断函数
$R_{(s,t)}$	(s, t) 的可达率
E	网络总体的期望可达率
$ (s, t) $	网络的交通需求数
$P_{(s,t)}^{short}$	最短路径集或较短路径集
$ P_{(s,t)}^{short} $	最终选定的从 s 到 t 的路径数
m	路段数
c_k	第 k 条路段的容量上限
β	每条路段出现突发状况的概率

注: 本文中默认对矩阵作加粗处理

五、模型的建立与求解

5.1 网络图与线性规划模型的建立

5.1.1 图论相关定义^[1]

定义赋权有向图 $D = (V, A, W)$, 其中 $V = \{v_1, v_2, \dots, v_n\}$ 为非空顶点集, $A =$

$\{a_1, a_2, \dots, a_m\}$ 为非空弧集合，每一条弧与一个有序的顶点对相对应，弧 $a_k = (v_i, v_j)$ 表示弧的方向自顶点 v_i 指向 v_j 。

$W = (w_{ij})_{n \times n}$ 定义为 D 的邻接矩阵，其中：

$$w_{ij} = \begin{cases} 1, & (v_i, v_j) \in A, \\ 0, & (v_i, v_j) \notin A, \end{cases} \quad i, j = 1, 2, \dots, n \quad (1)$$

设 $p = v_1 a_1 v_2 a_2 \dots v_k a_k v_{k+1}$ ，其中：

$$\begin{cases} v_i \in V, & i = 1, 2, \dots, k+1 \\ a_j \in A, & j = 1, 2, \dots, k \\ a_j = (v_j, v_{j+1}) \end{cases} \quad (2)$$

则称 p 是图 D 的一条路径。

需要说明的是，此时的 i 或 j 表示的是 v 与 a 在路径 p 中的前后对应关系，而非集合 V 与 A 中的第 i 或 j 个元素。

5.1.2 本题相关定义

与图论中严格定义相对应的，本题中“网络”与“赋权有向图”等价、“路径”定义不变、“路段”与“弧”等价，下文不再赘述。

现给定一(起点，终点)对 (s, t) ， $P_{(s,t)}$ 表示从起点 s 到终点 t 的所有可能路径，对于每一条路径 $p \in P_{(s,t)}$ ，定义 $x_p^{(s,t)}$ 为分配到路径 p 上的交通量。

对于每一个 (s, t) ，附件中都给出了一个对应的总交通量 $Total_{(s,t)}$ ，显然有：

$$Total_{(s,t)} = \sum_{p \in P_{(s,t)}} x_p^{(s,t)} \quad (3)$$

当某个路段 $a_k = (v_i, v_j)$ 出现突发状况时，若 $a_k \in p$ ，则 $x_p^{(s,t)} = 0$ ，我们不妨定义一个判断函数 y_p^i ，令：

$$y_p^i = \begin{cases} 0, & a_k \in p, \\ 1, & a_k \notin p, \end{cases} \quad i = 1, 2, \dots, n \quad (4)$$

其中， n 代表有 n 种突发状况的可能。

那么 (s, t) 的可达率 $R_{(s,t)}$ 可表示为：

$$R_{(s,t)} = \frac{\frac{1}{n} \sum_{i=1}^n \sum_{p \in P_{(s,t)}} x_p^{(s,t)} y_p^i}{Total_{(s,t)}} \quad (5)$$

总体的期望可达率 E 可表示为：

$$E = \frac{1}{|(s, t)|} \sum_{(s, t)} R_{(s, t)} \quad (6)$$

其中， $|(s, t)|$ 表示有 $|(s, t)|$ 个(起点，终点)对交通需求。

5.1.3 线性规划模型的建立

本题的核心是求解一个线性规划问题。

• 决策变量

在该模型中，我们定义分配到路径 p 上的交通量 $x_p^{(s, t)}$ 为决策变量。

• 目标函数

题目明确提到，本题求解的目标是让“网络中所有交通需求的期望可达率最大”，即让总体的期望可达率 E 最大。但在实际求解中，若不考虑路径长短， $P_{(s, t)}$ 的理论数量为无穷大，若令 p 遍历 $P_{(s, t)}$ ，也将产生无穷多个决策变量。

根据模型假设中的新增假设，在进行大规模网络求解时，我们以最短路径集或较短路径集 $P_{(s, t)}^{short}$ 替代 $P_{(s, t)}$ ，由此得出目标函数：

$$\max_{x_p^{(s, t)}} E = \frac{1}{|(s, t)|} \sum_{(s, t)} \frac{\frac{1}{n} \sum_{i=1}^n \sum_{p \in P_{(s, t)}^{short}} x_p^{(s, t)} y_p^i}{Total_{(s, t)}} \quad (7)$$

• 约束条件

1、交通量守恒

分配到每一条路径 p 上的交通量之和等于总交通量 $Total_{(s, t)}$ ：

$$Total_{(s, t)} = \sum_{p \in P_{(s, t)}} x_p^{(s, t)} \quad (8)$$

2、路径数约束

对于一个给定的 (s, t) ，最终选定的从 s 到 t 的路径数 $|P_{(s, t)}^{short}|$ 不能超过 5 条：

$$|P_{(s, t)}^{short}| \leq 5 \quad (9)$$

3、路段容量约束

假设网络中总共有 m 条路段，则对于每一条路段 $a_k = (v_i, v_j)$ ，其容量不能超过其容量上限 c_k ：

$$a_k \leq c_k, \quad k = 1, 2, \dots, m \quad (10)$$

4、非负约束

根据题给假设，决策变量 $x_p^{(s, t)}$ 可以为任意的非负实数：

$$x_p^{(s, t)} \geq 0 \quad (11)$$

• 完整表达

$$\max_{x_p^{(s,t)}} E = \frac{1}{|(s,t)|} \sum_{(s,t)} \frac{\frac{1}{n} \sum_{i=1}^n \sum_{p \in P_{(s,t)}^{short}} x_p^{(s,t)} y_p^i}{Total_{(s,t)}}$$

$$s. t. \begin{cases} Total_{(s,t)} = \sum_{p \in P_{(s,t)}} x_p^{(s,t)} \\ |P_{(s,t)}^{short}| \leq 5 \\ a_k \leq c_k, \quad k = 1, 2, \dots, m \\ x_p^{(s,t)} \geq 0 \end{cases} \quad (12)$$

5.2 对问题一的求解

5.2.1 对交通网络二的分析

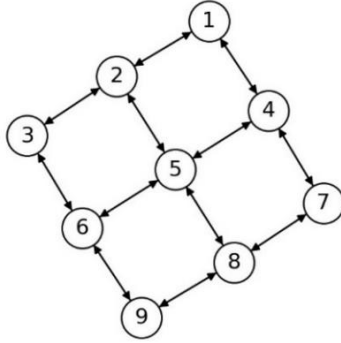


表 1 交通网络二信息

节点数	9 个
路段数	24 条
n	24 种
$ (s,t) $	36 个

图 1 交通网络二

图 1 显示了交通网络二的结构，表 1 显示了交通网络二的信息。

其中， n 表示交通网络二有 24 种突发状况的可能， $|(s,t)|$ 表示交通网络二有 36 个(起点，终点)对交通需求。

同时，注意到问题一无需考虑路段容量的约束。

至此，得出问题一的完整表达：

$$\max_{x_p^{(s,t)}} E = \frac{1}{|(s,t)|} \sum_{(s,t)} \frac{\frac{1}{24} \sum_{i=1}^{24} \sum_{p \in P_{(s,t)}^{short}} x_p^{(s,t)} y_p^i}{Total_{(s,t)}}$$

$$s. t. \begin{cases} Total_{(s,t)} = \sum_{p \in P_{(s,t)}} x_p^{(s,t)} \\ |P_{(s,t)}^{short}| \leq 5 \\ x_p^{(s,t)} \geq 0 \end{cases} \quad (13)$$

5.2.2 问题一线性规划求解

注意到，在式(13)中，除了决策变量 $x_p^{(s,t)}$ 外，其余都为可确定的常量，也就是说，式(13)最终可化简为以下形式：

$$\max_x E = f^T x$$

$$s. t. \begin{cases} A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub \end{cases} \quad (14)$$

式中， f^T 为决策变量 x 的系数向量， A 为不等约束的系数矩阵， Aeq 为等式约束的系数矩阵， lb/ub 分别对应了 x 的下界/上界，在本题中， x 下界为 0，上界为空。

贯穿问题 1 至 4 的核心任务就是完成从式(13)到式(14)的化简，及式(14)的线性规划求解。

• 较短路径法

下表给出了计算问题一目标函数系数向量 f^T 的伪代码：

表 2 问题一求解 f^T 算法 基于 Python

伪代码 1：计算问题一目标函数系数向量 f^T	
Step1：数据初始化	
1:	根据交通网络二构建邻接矩阵 $W_1^{9 \times 9}$ ：存储道路畅通样本；
2:	读取 $W_1^{9 \times 9}$ 中“1”的坐标，存储为集合 Set_1 ；
3:	基于 $W_1^{9 \times 9}$ 构建赋权有向图 D_1 ；
4:	构建 $W_2^{9 \times 9}, Set_2, D_2$ ：存储突发状况样本；
5:	赋值随机采样次数 $N = 24/100/1000/10000$ ；
6:	赋值每次采样样本 $S = 1$ (随机令 1 条路段出现突发状况).
Step2：标记道路畅通样本	
7:	for $i = 1: (s, t) $
8:	给定 (s, t) ；
9:	根据 $nx.all_simple_path$ 函数查找 D_1 中 s 到 t 的所有无环路径；
10:	将无环路径降序排列，取最短 5 条添加到 $path_1$ 中；
11:	将 $path_1$ 中的 5 条路径标记为变量 $x_{5i-4}: x_{5i}$ ；
Step3：添加随机扰动	
12:	for $j = 1: N$
13:	初始化 $W_2^{9 \times 9} = W_1^{9 \times 9}, Set_2 = Set_1, D_2 = D_1$ ；
14:	随机将 $W_2^{9 \times 9}$ 中 S 个坐标从“1”变为“0”，更新 Set_2, D_2 ；
Step4：标记突发状况样本	
15:	根据 $nx.all_simple_path$ 函数查找 D_2 中 s 到 t 的所有无环路径；
16:	将无环路径降序排列，取最短 5 条添加到 $path_2$ 中；
17:	$path_2$ 对照 $path_1$ 作遍历循环，查找共同路径；
18:	将共同路径的对应标记集 $\{x\}$ 自增 1.
19:	end
20:	令 $\{x\} = \{x\}/N$ ；
21:	令 $\{x\} = \{x\}/Total(s, t)$.
22:	end
23:	令 $\{x\} = \{x\}/ (s, t) $ ；
24:	Result: $\{x\}^T$ 即为 f^T .

注意到在上述算法中，我们并没有采取遍历 24 种突发状况可能的方式，而是每次随机令一条路段出现突发状况，如此采样 $N = 24/100/1000/10000$ 次，再取平均值。虽然这样做可能会采样到重复的路段，但当 N 的数值远超 24 时， f^T 将逐渐收敛。

还有一点值得注意的是，我们 $P_{(s,t)}^{short}$ 中的 5 条最短路径，并非真正意义上的最短路径，例如：对于(起点，终点)对(6,22)，我们寻得的路径为：

$$\begin{cases} 6 \rightarrow 5 \rightarrow 22 \\ 6 \rightarrow 34 \rightarrow 22 \\ 6 \rightarrow 34 \rightarrow 40 \rightarrow 22 \\ 6 \rightarrow 36 \rightarrow 16 \rightarrow 5 \rightarrow 22 \\ 6 \rightarrow 0 \rightarrow 3 \rightarrow 36 \rightarrow 16 \rightarrow 5 \rightarrow 22 \end{cases} \quad (15)$$

我们只能姑且称其为“较短路径集”。但这样做的好处是，既为后续MATLAB的求解带来了参数上的便利，又隐含地完成了 $|P_{(s,t)}^{short}| \leq 5$ 的不等约束，这样在接下来调用linprog函数求解线性规划时，就只需考虑等式约束与非零约束了：

表 3 最大化期望可达率算法 基于MATLAB

伪代码 2：最大化问题一期望可达率

- 1: 导入数据;
- 2: $[x, fval] = \text{linprog}(f^T, [], [], Aeq, beq, lb);$
- 3: 可视化 $fval$.

最终，我们求得以下数据：

表 4 较短路径集期望可达率

随机采样数 N	期望可达率 E
24	90.51%
100	89.63%
1000	89.04%
10000	88.96%

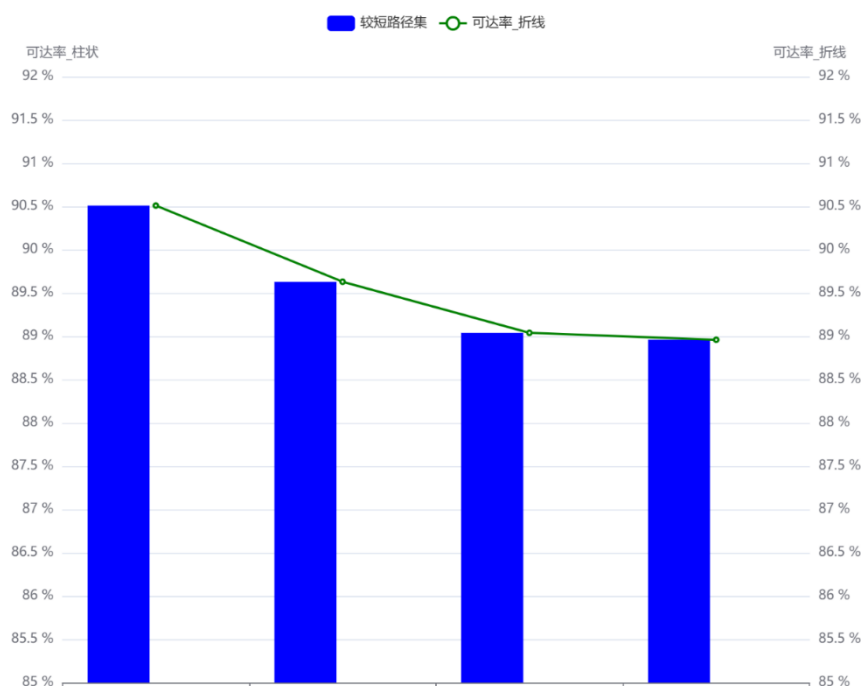


图 2 较短路径集期望可达率

可以看出,随着随机采样数 N 的不断增加,期望可达率 E 将逐渐收敛于89%附近。

• 最短路径法

我们注意到,相较于交通网络三,交通网络二并没有那么复杂,我们可以通过人工筛选的方式,构建出下表所示的“最短路径集”,进而探索在最短路径下是否存在更高的期望可达率:

表 5 交通网络二最短路径集

(s, t)	最短路径集					
$(1, 5)$	1-2-5	1-4-5				
$(1, 6)$	1-2-3-6	1-2-5-6	1-4-5-6			
$(1, 8)$	1-4-7-8	1-4-5-8	1-2-5-8			
$(1, 9)$	1-2-3-6-9	1-4-7-8-9	1-2-5-8-9	1-4-5-6-9	1-2-5-6-9	1-4-5-8-9
...	...	...	...	...	...	...
$(9, 1)$	9-8-7-4-1	9-8-5-2-1	9-8-5-4-1	9-6-3-2-1	9-6-5-4-1	9-6-5-2-1
$(9, 2)$	9-6-3-2	9-8-5-2	9-6-5-2			
$(9, 4)$	9-6-5-4	9-8-7-4	9-8-5-4			
$(9, 5)$	9-8-5	9-6-5				

其中,由于 $|P_{(s,t)}^{short}| \leq 5$ 的约束,我们需要从有 6 条最短路径的 (s, t) 中划去一条,即表中黄色的部分。

我们注意到,此时若 (s, t) 只有两条最短路径,便只选取这两条作为我们的 $P_{(s,t)}^{short}$,若有三条,则选取三条...,符合现实意义中最短路径集的定义。

我们分别给定 $\{1 \rightarrow 2, 2 \rightarrow 1, \dots, 8 \rightarrow 9, 9 \rightarrow 8\}$ 共 24 种突发状况的可能,在表 5 的基础上绘制出 24 张新表。将受到突发状况影响的路径置“0”,将不受突发状况影响的路径置“1”,共形成 24 个 36×5 的“01”矩阵,将这 24 个矩阵求和,得到如下遍历矩阵:

表 6 最短路径集遍历矩阵

22	22	0	0	0
21	21	21	0	0
22	22	22	0	0
20	21	21	20	20
...	...	...	...	...
21	21	21	20	20
21	22	21	0	0
21	22	22	0	0
23	22	0	0	0

将表 6 带入目标函数与等式约束求得以下数据:

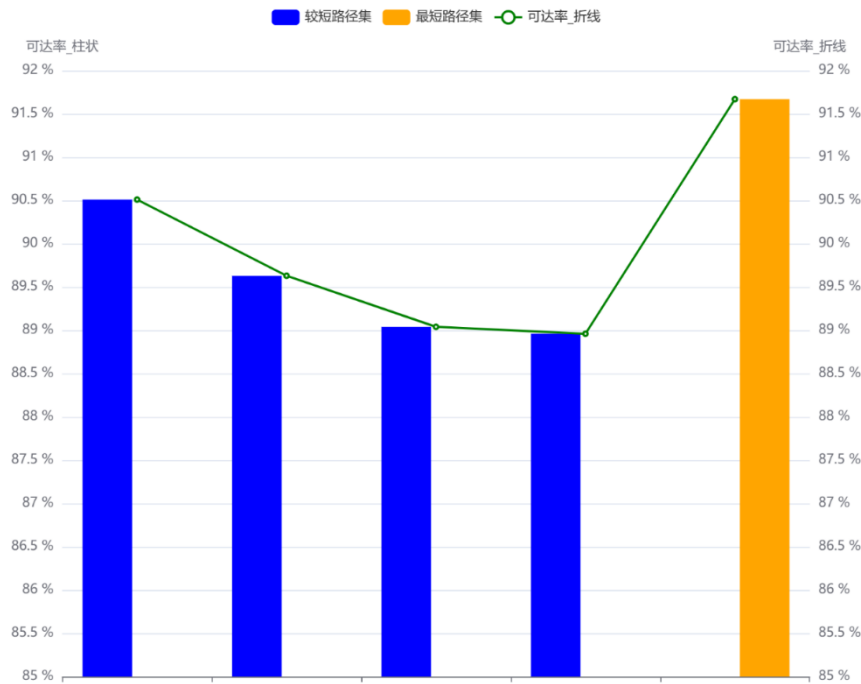


图 3 最短路径集期望可达率

此时，期望可达率 E 来到了91.67%，相比较短路径集有了显著的提升，对于这种现象我们不妨进行以下讨论：

假设每条路段出现突发状况的概率为 β ，那么每条路段不出现突发状况的概率为 $1 - \beta$ 。假设给定一(起点，终点)对 (s, t) ，从 s 到 t 共经过路段 m 条，那么这 m 条路段都不出现突发状况的概率为 $(1 - \beta)^m$ 。

显然， $1 - \beta$ 是一个小于1的常数， m 越大，这 m 条路段都不出现突发状况的概率越低，同时我们又知道，当这 m 条路段中任意一条路段出现突发状况时， $x_p^{(s,t)}$ 将置“0”。也就是说，从 s 到 t 的路径越长，受到突发状况干扰的可能性越大，期望可达率越低，要想提高期望可达率，就需要尽可能地缩短从 s 到 t 的路径长度。

这也就说明了为什么从最短路径集上遍历得到的期望可达率要高于较短路径集，这一点认识对于我们求解问题四至关重要。

至此，我们给出问题一的结果：

表 7 问题一结果

(起点,终点)	规划路径	分配交通量
(1,9)	1-4-7-8-9	180
(3,7)	3-2-1-4-7	140

5.3 对问题二的求解

5.3.1 对交通网络三的分析

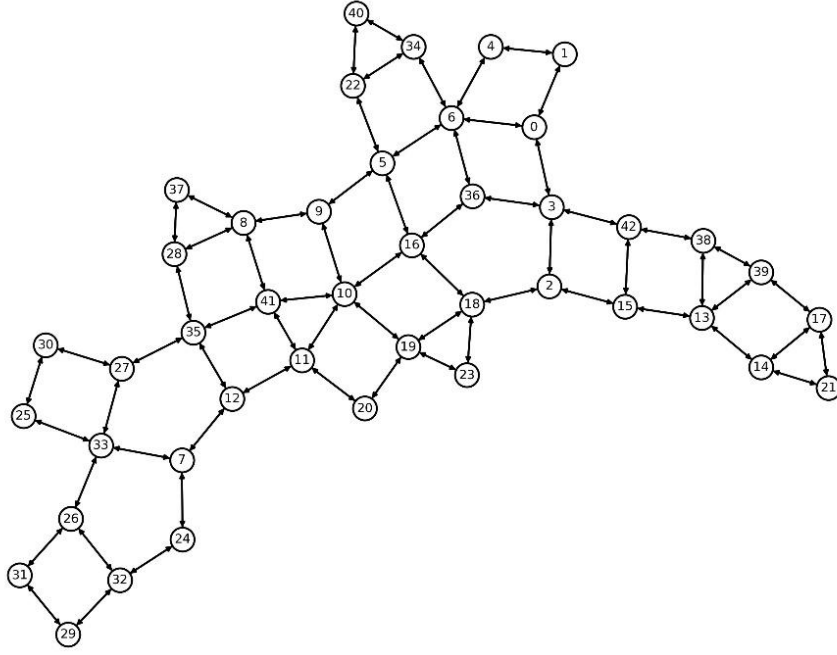


图 4 交通网络三

表 8 交通网络三信息

节点数	43 个
路段数	132 条
n	C_{132}^5 种
$ (s, t) $	100 个

图 4 显示了交通网络三的结构，表 8 显示了交通网络三的信息。我们不难给出问题二的完整表达：

$$\begin{aligned} \max_{x_p^{(s,t)}} E &= \frac{1}{|(s, t)|} \sum_{(s,t)} \frac{\frac{1}{C_{132}^5} \sum_{i=1}^{C_{132}^5} \sum_{p \in P_{(s,t)}^{short}} x_p^{(s,t)} y_p^i}{Total_{(s,t)}} \\ s. t. &\begin{cases} Total_{(s,t)} = \sum_{p \in P_{(s,t)}} x_p^{(s,t)} \\ |P_{(s,t)}^{short}| \leq 5 \\ x_p^{(s,t)} \geq 0 \end{cases} \end{aligned} \quad (16)$$

5.3.2 问题二线性规划求解

注意到，相比于交通网络二，交通网络三在各项指标上都有了显著的提升，突发状况的可能数更是高达 $C_{132}^5 = 3,0931,9296$ 种。

正是由于 C_{132}^5 这种超高参数的限制，我们无法再像问题一一样通过人工筛选构建最短路径集，只能使用基于伪代码 1 的较短路径集求解 f^T 。

这里只给出问题二算法相较于问题一算法的不同之处：

表 9 问题二求解 f^T 算法 基于 Python

伪代码 3：计算问题二目标函数系数向量 f^T

Step1: 数据初始化

- 1: 将 $W_1^{9 \times 9}, W_2^{9 \times 9}$ 更改为 $W_1^{43 \times 43}, W_2^{43 \times 43}$;
- 2: 赋值随机采样次数 $N = 10/100/1000$;
- 3: 赋值每次采样样本 $S = 5$ (随机令 5 条路段出现突发状况).

依旧是将 f^T 带入 $MATLAB$ 求得以下数据:

表 10 问题二期望可达率

随机采样数 N	期望可达率 E
10	85.34%
100	82.14%
1000	81.53%

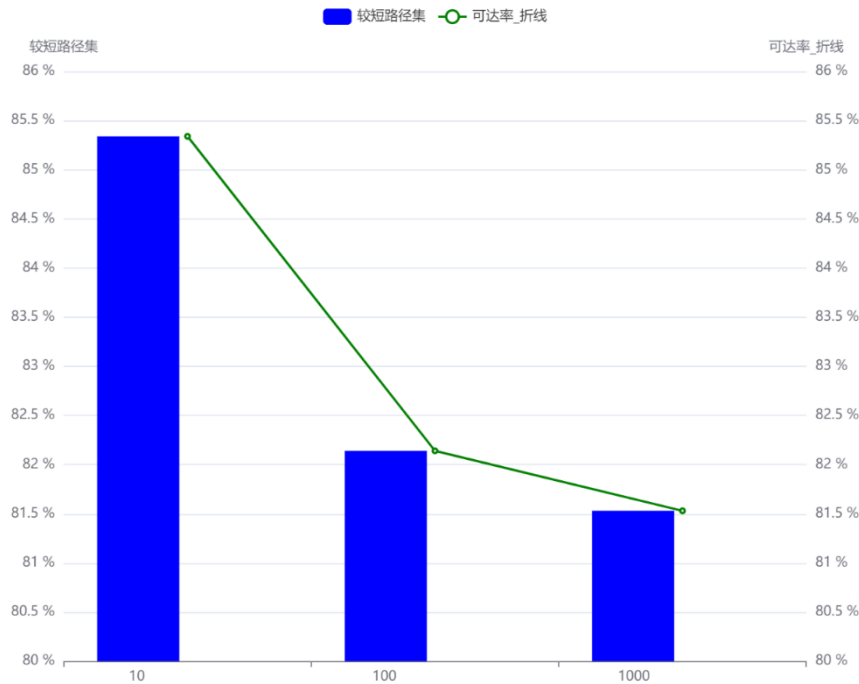


图 5 问题二期望可达率

需要说明的是, 这里随机采样数 N 最高取 1000 是因为我们的计算机无法承担更大的运行开销, 若我们拥有更强的算力或更长的时间, 可进一步提高 N 的取值。但从有限的数据中依然可以看出, 随着 N 的提高, 期望可达率向81%附近收敛。

至此, 我们给出问题二的结果:

表 11 问题二结果

(起点,终点)	规划路径	分配交通量
(27,6)	27-35-28-8-9-5-6	121
(19,25)	19-10-41-35-27-30-25	115

5.4 对问题三的求解

5.4.1 路段约束到路径约束的转换

问题三相较于问题二, 只是给附件 3 中提及的路段增添了容量上限, 我们不难给出问题三的完整表达:

$$\begin{aligned} \max_{x_p^{(s,t)}} E &= \frac{1}{|(s,t)|} \sum_{(s,t)} \frac{\frac{1}{C_{132}^5} \sum_{i=1}^{C_{132}^5} \sum_{p \in P_{(s,t)}^{short}} x_p^{(s,t)} y_p^i}{Total_{(s,t)}} \\ s.t. &\begin{cases} Total_{(s,t)} = \sum_{p \in P_{(s,t)}} x_p^{(s,t)} \\ |P_{(s,t)}^{short}| \leq 5 \\ a_k \leq c_k, \quad k = 1, 2, \dots, m \\ x_p^{(s,t)} \geq 0 \end{cases} \end{aligned} \quad (17)$$

但困难的是，在我们的线性规划模型中路径是决策变量，所以在正式求解前，我们先得完成从路段约束到路径约束的转换。

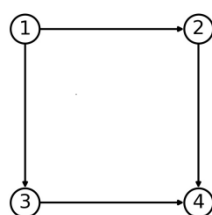


图 6 交通网络一

让我们回顾一下题目最开始给出的交通网络一，当给路径 1-2-4 分配 40 辆车时，路段 1-2 和路段 2-4 的交通量都会增加 40，也就是说，当不同路径经过同一路段时，其交通量将会在该路段上叠加。

由于我们的算法对于给定的 (s, t) 每次会生成 5 条较短路径，而在附件 2 中共有 100 条交通需求，从而一共会生成 500 条较短路径，我们将这 500 条路径从 1 至 500 依次编号，用于构建不等约束的系数矩阵。

我们需要根据附件 3，从伪代码 3 生成的较短路径集 $\sum_{(s,t)} P_{(s,t)}^{short}$ 中寻找哪些路径经过了路段 $a_k (k = 1, 2, \dots, 132)$ ：

表 12 经过 $a_k (k = 1, 2, \dots, 132)$ 的路径编号

路段起点	路段终点	路段容量	路径编号
0	1	315	9,306,307,309,331, ...,401
0	3	393	15,26,27,29,30,201, ...,359
0	6	370	1,3,5,136,138,140, ...,464
...	...	...	...
42	38	460	26,27,28,105,183, ...,495

再根据 a_k 对应的路径编号，将不等约束系数矩阵 $A = (A_{ij})_{132 \times 500}$ 的对应位置置“1”，例如，黄色方块表示路径 15 经过路段 a_2 ：

表 13 不等约束系数矩阵

1	...	15	...	306	...	495	...	500
---	-----	----	-----	-----	-----	-----	-----	-----

a_1	0	...	0	...	1	...	0	...	0
a_2	0	...	1	...	0	...	0	...	0
a_3	1	...	0	...	0	...	0	...	0
...	...	...	...	...	...	...	...	...	...
a_{132}	0	...	0	...	0	...	1	...	0

5.4.2 问题三线性规划求解

最终，将新增的不等约束带入线性规划方程，求得问题三的结果：

表 14 问题三结果

(起点,终点)	规划路径	分配交通量
(32,39)	32-24-7-12-11-10-16-18-2-15-13-39	65
(17,8)	17-14-13-15-2-18-16-10-9-8	63

5.5 对问题四的求解

5.5.1 判断最有价值路径

在交通网络三中，共有 24 条环，其中边数为 3 的环有 6 条，边数为 4 的环有 15 条，边数为 5 的环有 3 条。

对于边数为 3 的环，我们无法在其内部新修路段；对于边数为 4 的环，我们可在其内部新修 4 条路段；对于边数为 5 的环，我们可在其内部新修 10 条路段。

由此统计，在交通网络三中，有 90 种新修路段的可能，若每次从中新修 6 条，则共有 $C_{90}^6 = 6,2261,4630$ 种可能，且还要判断这 6 条路段是否存在交叉，这样求解问题显得过于复杂。

于是，我们可以通过人工筛选的方式，将“没有价值”的路段剔除，将“最有价值”的路段保留，以此缩小我们的遍历集。

我们是这样做的：

根据附件 2，统计各节点作为起点和终点的次数，并将二者求和：

表 15 附件 2 中各节点作为起点和终点的次数

$v_i (i = 1, 2, \dots, 42)$	起点次数	终点次数	总计
23	6	5	11
17	5	5	10
40	3	6	9
13	4	3	7
20	1	6	7
21	5	2	7
...	...	...	...
42	0	0	0

我们观察到，节点 v_{23}, v_{17}, v_{40} 在起点和终点的总计上有明显优势，也就是说，它们将更频繁地参与到交通需求的运输中。优化了它们，也就等于最大效率上优化

了网络总体的期望可达率。
那么，我们该如何定义优化的方向呢？

5.5.2 以最短路径为目标

在问题一的求解中我们曾经证明过，从 s 到 t 的路径越长，受到突发状况干扰的可能性越大，期望可达率越低，要想提高期望可达率，就需要尽可能地缩短从 s 到 t 的路径长度。

也就是说，如果我们能够利用这 6 条新修路段帮助节点 v_{23}, v_{17}, v_{40} 缩短其作为起点、终点的路径长度，那么就变向等于优化了网络总体的期望可达率。

表 16 v_{23}, v_{17}, v_{40} 作为起点		表 17 v_{23}, v_{17}, v_{40} 作为终点	
起点	终点	起点	终点
23	24	35	23
23	10	32	23
...	...	...	...
17	6	27	17
17	29	27	17
...	...	...	...
40	27	28	40
40	8	27	40
...	...	...	...

根据附件 2，我们能够列出 v_{23}, v_{17}, v_{40} 分别作为起点与终点时的对应 (s, t) ，我们通过这些 (s, t) 连接起来，并通过新修路段的方式进一步缩短这些 (s, t) 的路径长度，将成功缩短路径长度的新修路段以单向线的形式在图中标出：

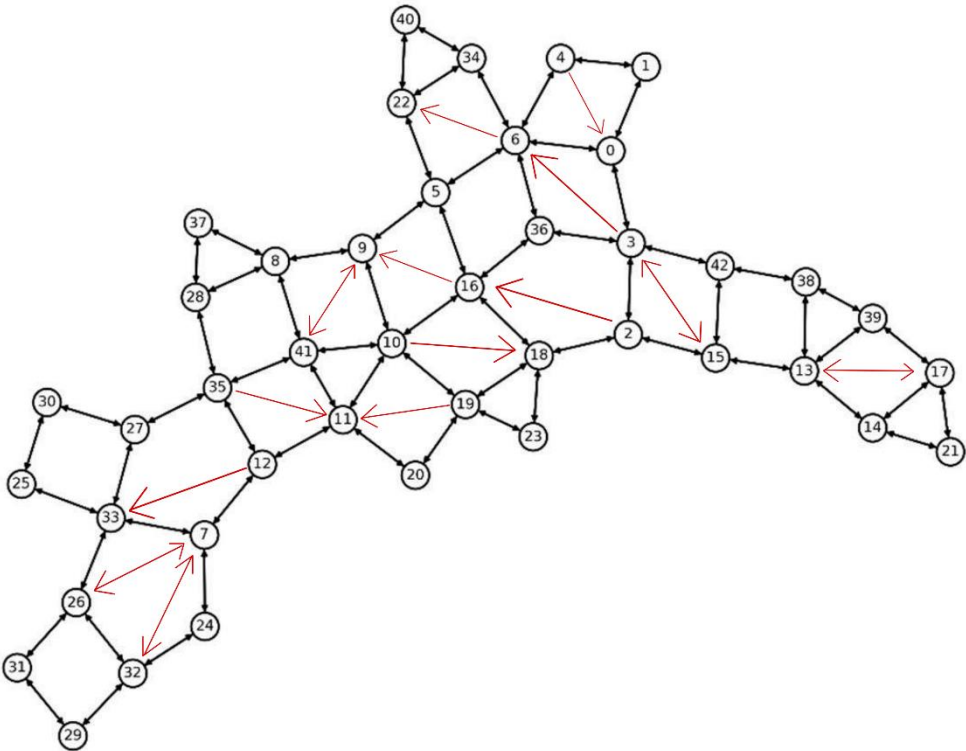


图 7 “最价值” 新修路段集

表 18 “最有价值”新修路段编号

起点	终点	续上	
19	11	15	3
7	26	4	0
12	33	10	18
7	32	3	15
32	7	13	17
35	11	9	41
3	6	41	9
17	13	41	9
17	13	26	7
2	16	15	3
16	9	3	6
续下		6	22

其中我们发现，节点对(3,6), (17,13), (15,3), (41,9)在连接路径时被反复用到，也就是说，在这些位置新修路段，能帮助更多的路径缩短长度，类似于交通网络中的“枢纽”。我们选择将这些节点对固定下来，作为新修路段 6 条中的 4 条，再从剩下的 15 条中随机抽取两条，组成我们最终的 6 条新修路段。这样一来，我们的遍历组合便从最初的 $C_{90}^6 = 6,2261,4630$ 大幅降低到了 $C_{15}^2 = 105$ ，在减少运行开销的同时，也保证了不会因减少遍历组合数而导致期望可达率降低。

5.5.3 问题四线性规划求解

我们将采样结果带入到线性规划方程，求得问题四的结果：

表 19 问题四结果

	新建路段 1	新建路段 2	新建路段 3	新建路段 4	新建路段 5	新建路段 6	可达率
方案 1	7-26	19-11	3-6	17-13	15-3	41-9	92.97%
方案 2	16-9	19-11	3-6	17-13	15-3	41-9	92.71%
方案 3	10-18	7-26	3-6	17-13	15-3	41-9	90.55%
方案 4	3-6	19-11	3-6	17-13	15-3	41-9	89.82%
方案 5	15-3	6-22	3-6	17-13	15-3	41-9	88.95%

六、参考文献

[1] 司守奎, 孙玉玺. 《数学建模算法与应用》. 北京: 国防工业出版社, 2023.

七、附录

#Python代码——第 1 问 9 阶矩阵

```
import networkx as nx
import pandas as pd
import random
import itertools
import numpy as np # 如果需要导入为 NumPy 数组
```

```

from collections import defaultdict

# 读取 Excel 文件，提取涉及的起点和终点
file_path_traffic = r"C:\Users\31105\Desktop\附件 1 (Attachment 1)-交通需求 (交通网络 2)Traffic demand (Network 2).xlsx"
df_traffic = pd.read_excel(file_path_traffic)

# 提取所有涉及的起点和终点
origins = df_traffic.iloc[:, 0].unique()
destinations = df_traffic.iloc[:, 1].unique()

# 读取 9x9 矩阵，构建网络
file_path_matrix = r"C:\Users\31105\Desktop\33 矩阵.xlsx"
original_matrix = pd.read_excel(file_path_matrix, header=None)

# 定义循环次数和每次选择的样本数
num_iterations = 10000 # 循环次数
num_samples_per_iteration = 1 # 每次循环选择的样本数

# 全局变量，用于记录共同路径计数
common_path_counts = [defaultdict(int) for _ in range(180)]

# 初始化路径标号
path_id_counter = itertools.count(1)

# 全局变量，用于记录路径标号的累加计数
path_id_counts = defaultdict(int)

# 检查是否满足条件
if num_samples_per_iteration > len(original_matrix):
    raise ValueError(f"采样次数 ({num_samples_per_iteration}) 超过了可用的坐标数量 ({len(original_matrix)})")

# 循环执行
for iteration in range(num_iterations):
    print(f"Iteration {iteration + 1}:")

    # 初始化矩阵
    df_matrix = original_matrix.copy()

    # 集合来存储所有数值为 1 的坐标
    one_coordinates = set()
    for row_idx in range(df_matrix.shape[0]):
        for col_idx in range(df_matrix.shape[1]):

```

```

        if df_matrix.iat[row_idx, col_idx] == 1:
            one_coordinates.add((row_idx, col_idx))

# 随机选择
random_positions = random.sample(list(one_coordinates),
num_samples_per_iteration)

print("Randomly selected positions:")
for pos in random_positions:
    print(pos)

# 将这些坐标设为 0
for pos in random_positions:
    row, col = pos
    df_matrix.iat[row, col] = 0

# 创建两个图
G_1 = nx.DiGraph()
G_2 = nx.DiGraph()

# 遍历矩阵, 添加边
for i in range(9):
    for j in range(9):
        if df_matrix.iloc[i, j] == 1:
            G_1.add_edge(i + 1, j + 1)
        if original_matrix.iloc[i, j] == 1:
            G_2.add_edge(i + 1, j + 1) # `G_2` 为原始路线

# 找到每一行对应的起点和终点之间的所有路径
demand_distribution_1 = {}
demand_distribution_2 = {}

for row in df_traffic.itertuples():
    origin = row[1]
    destination = row[2]

    if origin != destination:
        all_paths_1 = list(nx.all_simple_paths(G_1, source=origin,
target=destination))
        all_paths_2 = list(nx.all_simple_paths(G_2, source=origin,
target=destination))

# 为每条路径添加唯一标号
all_paths_sorted_1 = [

```

```

        (next(path_id_counter), path)
        for path in sorted(all_paths_1, key=len)
    ]
    all_paths_sorted_2 = [
        (next(path_id_counter), path)
        for path in sorted(all_paths_2, key=len)
    ]

    demand_distribution_1[(origin, destination)] =
all_paths_sorted_1[:5]
    demand_distribution_2[(origin, destination)] =
all_paths_sorted_2[:5]

# 找到相同路径并为它们的标号计数
common_paths = {}
for key in demand_distribution_1:
    if key in demand_distribution_2:
        common = [
            path for _, path in demand_distribution_1[key]
            if path in [p for _, p in demand_distribution_2[key]]
        ]
        if common:
            common_paths[key] = common # 当前循环中找到的路径

            # 累加 `path_id_counts` 中的标号
            for path in common:
                path_tuple = tuple(path)
                path_id_counts[path_tuple] += 1

# 输出相同的路径及其计数
for (origin, destination), paths in common_paths.items():
    print(f"Common paths from {origin} to {destination}:")
    for path in paths:
        path_str = " -> ".join(map(str, path))
        print(f"{path_str} (Count: {path_id_counts[tuple(path)]})")

print() # 换行符, 方便区分循环输出

j = 0
# 循环结束后, 汇总输出所有路径的计数
print("Summary of all path counts (按表格顺序):")

# 遍历 `df_traffic` 中的起点和终点
for row in df_traffic.itertuples():

```

```

j = j+1
origin = row[1]
destination = row[2]
demand = row[3]

# 获取第三列
# Pandas DataFrame 的列是基于零的索引，所以第三列的索引是 2
third_column = df_traffic.iloc[:, 2]
# 转换为列表
third_column_list = third_column.tolist() # 表格中的第三列-demand

# 检索与当前起点和终点相关的路径
key = (origin, destination)
if key in demand_distribution_2: # G_2 中的路径
    paths = demand_distribution_2[key]
    for _, path in paths:
        path_tuple = tuple(path)
        path_str = " -> ".join(map(str, path_tuple))
        if path_tuple in path_id_counts:
            count = path_id_counts[path_tuple]
            print(f"{count / third_column_list[j - 1] / 36 /
num_iterations} ")
# 调试代码-输出更好看，直观      print(f"{count/third_column_list[j-
1]/36/num_iterations} --对应的 demand 为多少{third_column_list[j-1]}") # 在
随机取样中的累计总次数/列表 demand/循环次数/

print(f"{j}")

```

#Python代码——第 2 问 43 阶矩阵

```

import networkx as nx
import pandas as pd
import random
import itertools
import numpy as np # 如果需要导入为 NumPy 数组
from collections import defaultdict

# 读取 Excel 文件，提取涉及的起点和终点
file_path_traffic = r"C:\Users\31105\Desktop\附件 2 (Attachment 2)-交通需求
(交通网络 3)Traffic demand (Network 3).xlsx"
df_traffic = pd.read_excel(file_path_traffic)

# 提取所有涉及的起点和终点
origins = df_traffic.iloc[:, 0].unique()
destinations = df_traffic.iloc[:, 1].unique()

```

```

# 读取 9x9 矩阵, 构建网络
file_path_matrix = r"C:\Users\31105\Desktop\邻接矩阵.xlsx"
original_matrix = pd.read_excel(file_path_matrix, header=None)

# 定义循环次数和每次选择的样本数
num_iterations = 1000 # 循环次数
num_samples_per_iteration = 5 # 每次循环选择的样本数

# 全局变量, 用于记录共同路径计数
common_path_counts = [defaultdict(int) for _ in range(180)]

# 初始化路径标号
path_id_counter = itertools.count(1)

# 全局变量, 用于记录路径标号的累加计数
path_id_counts = defaultdict(int)

# 检查是否满足条件
if num_samples_per_iteration > len(original_matrix):
    raise ValueError(f"采样次数 ({num_samples_per_iteration}) 超过了可用的坐标数量 ({len(original_matrix)})")

# 循环执行
for iteration in range(num_iterations):
    print(f"Iteration {iteration + 1}:")

    # 初始化矩阵
    df_matrix = original_matrix.copy()

    # 集合来存储所有数值为 1 的坐标
    one_coordinates = set()
    for row_idx in range(df_matrix.shape[0]):
        for col_idx in range(df_matrix.shape[1]):
            if df_matrix.iat[row_idx, col_idx] == 1:
                one_coordinates.add((row_idx, col_idx))

    # 随机选择
    random_positions = random.sample(list(one_coordinates),
num_samples_per_iteration)

    # print("Randomly selected positions:")
    # for pos in random_positions:
    #     print(pos)

```

```

# 将这些坐标设为 0
for pos in random_positions:
    row, col = pos
    df_matrix.iat[row, col] = 0

# 创建两个图
G_1 = nx.DiGraph()
G_2 = nx.DiGraph()

# 遍历矩阵, 添加边
for i in range(43):
    for j in range(43):
        if df_matrix.iloc[i, j] == 1:
            G_1.add_edge(i, j)
        if original_matrix.iloc[i, j] == 1:
            G_2.add_edge(i, j) # `G_2` 为原始路线

# 找到每一行对应的起点和终点之间的所有路径
demand_distribution_1 = {}
demand_distribution_2 = {}

for row in df_traffic.itertuples():
    origin = row[1]
    destination = row[2]

    if origin != destination:
        # 限制路径的最大长度
        cutoff = 15 # 设置最大路径长度
        all_paths_1 = list(nx.all_simple_paths(G_1, source=origin,
target=destination, cutoff=cutoff))
        all_paths_2 = list(nx.all_simple_paths(G_2, source=origin,
target=destination, cutoff=cutoff))

        # 为每条路径添加唯一标号
        all_paths_sorted_1 = [
            (next(path_id_counter), path)
            for path in sorted(all_paths_1, key=len)
        ]
        all_paths_sorted_2 = [
            (next(path_id_counter), path)
            for path in sorted(all_paths_2, key=len)
        ]

```

```

        demand_distribution_1[(origin, destination)] =
all_paths_sorted_1[:5]
        demand_distribution_2[(origin, destination)] =
all_paths_sorted_2[:5]

# 找到相同路径并为它们的标号计数
common_paths = {}
for key in demand_distribution_1:
    if key in demand_distribution_2:
        common = [
            path for _, path in demand_distribution_1[key]
            if path in [p for _, p in demand_distribution_2[key]]
        ]
        if common:
            common_paths[key] = common # 当前循环中找到的路径

# 累加 `path_id_counts` 中的标号
for path in common:
    path_tuple = tuple(path)
    path_id_counts[path_tuple] += 1

# 输出相同的路径及其计数
for (origin, destination), paths in common_paths.items():
#     print(f"Common paths from {origin} to {destination}:")
    for path in paths:
        path_str = " -> ".join(map(str, path))
#         print(f"{path_str} (Count: {path_id_counts[tuple(path)]})")

    print() # 换行符, 方便区分循环输出

j = 0
# 循环结束后, 汇总输出所有路径的计数
print("Summary of all path counts (按表格顺序):")

# 遍历 `df_traffic` 中的起点和终点
for row in df_traffic.itertuples():
    j = j+1
    origin = row[1]
    destination = row[2]
    demand = row[3]

# 获取第三列
# Pandas DataFrame 的列是基于零的索引, 所以第三列的索引是 2
third_column = df_traffic.iloc[:, 2]

```



```

# 转换为列表
third_column_list = third_column.tolist() # 表格中的第三列-demand

# 检索与当前起点和终点相关的路径
key = (origin, destination)

if key in demand_distribution_2: # G_2 中的路径
    paths = demand_distribution_2[key]
    for _, path in paths:
        path_tuple = tuple(path)
        path_str = " -> ".join(map(str, path_tuple))
        if path_tuple in path_id_counts:
            count = path_id_counts[path_tuple]
            print(f"{count / third_column_list[j - 1] / 100 /
num_iterations} ")
# 调试代码-输出更好看, 直观      print(f"{count/third_column_list[j-
1]/36/num_iterations} --对应的 demand为多少{third_column_list[j-1]}") # 在
随机取样中的累计总次数/列表 demand/循环次数/

print(f"{j}")

```

%MATLAB代码——第 1 问 9 阶矩阵

```
clear,clc,close all
```

```

f_24 = table2array(readtable('data99.xlsx','Range','K2:K181'));
f_24 = -f_24;
f_100 = table2array(readtable('data99.xlsx','Range','V2:V181'));
f_100 = -f_100;
f_1k = table2array(readtable('data99.xlsx','Range','AG2:AG181'));
f_1k = -f_1k;
f_1w = table2array(readtable('data99.xlsx','Range','AR2:AR181'));
f_1w = -f_1w;

```

```

Aeq = zeros(36,180);
for i = 1:36
    Aeq(i,5*i-4:5*i) = 1;
end

```

```
beq = table2array(readtable('data1.xlsx','Range','C2:C37'));
```

```

[x_24,fval_24] = linprog(f_24,[],[],Aeq,beq,zeros(180,1));
[x_100,fval_100] = linprog(f_100,[],[],Aeq,beq,zeros(180,1));
[x_1k,fval_1k] = linprog(f_1k,[],[],Aeq,beq,zeros(180,1));
[x_1w,fval_1w] = linprog(f_1w,[],[],Aeq,beq,zeros(180,1));

```

```
fval_24 = -fval_24;  
fval_100 = -fval_100;  
fval_1k = -fval_1k;  
fval_1w = -fval_1w;
```

```
disp("可达率为: ")  
disp(fval_24)  
disp(fval_100)  
disp(fval_1k)  
disp(fval_1w)
```

%MATLAB代码——第2问43阶矩阵

```
clear,clc,close all
```

```
f_10 = table2array(readtable('data4343.xlsx','Range','F2:F501'));  
f_10 = -f_10;  
f_100 = table2array(readtable('data4343.xlsx','Range','L2:L501'));  
f_100 = -f_100;  
f_1k = table2array(readtable('data4343.xlsx','Range','R2:R501'));  
f_1k = -f_1k;
```

```
Aeq = zeros(100,500);  
for i = 1:100  
    Aeq(i,5*i-4:5*i) = 1;  
end
```

```
beq = table2array(readtable('data2.xlsx','Range','C2:C101'));
```

```
[x_10,fval_10] = linprog(f_10,[],[],Aeq,beq,zeros(500,1));  
[x_100,fval_100] = linprog(f_100,[],[],Aeq,beq,zeros(500,1));  
[x_1k,fval_1k] = linprog(f_1k,[],[],Aeq,beq,zeros(500,1));
```

```
fval_10 = -fval_10;  
fval_100 = -fval_100;  
fval_1k = -fval_1k;
```

```
disp("可达率为: ")  
disp(fval_10)  
disp(fval_100)  
disp(fval_1k)
```